

EXP NO: 1	SETTING UP THE ENVIRONMENT AND PREPROCESSING THE DATA
------------------	--

AIM:

To set up a fully functional machine learning development environment and to perform data preprocessing operations like handling missing values, encoding categorical variables, feature scaling, and splitting datasets.

Code1:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = sns.load_dataset('titanic')

# Data exploration
print(df.info())
print(df.describe())
print(df.isnull().sum())

# Handle missing values
df['age'] = df['age'].fillna(df['age'].median())
df['embark_town'] = df['embark_town'].fillna(df['embark_town'].mode()[0])
df.drop(columns=['deck'], inplace=True)

# Encode categorical variables
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
df['embark_town'] = le.fit_transform(df['embark_town'])

# Drop non-informative / redundant columns
df.drop(columns=['embarked', 'class', 'who', 'alive', 'adult_male', 'alone'], inplace=True)

# Feature Scaling
scaler = StandardScaler()
numerical_cols = ['age', 'fare']

df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# Train-Test Split
X = df.drop('survived', axis=1)
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training Data Shape:", X_train.shape)
print("Test Data Shape:", X_test.shape)
```

Output1:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
...
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   survived              891 non-null    int64
1   pclass                891 non-null    int64
2   sex                   891 non-null    object
3   age                   714 non-null    float64
4   sibsp                 891 non-null    int64
5   parch                 891 non-null    int64
6   fare                  891 non-null    float64
7   embarked              889 non-null    object
8   class                 891 non-null    category
9   who                   891 non-null    object
10  adult_male            891 non-null    bool
11  deck                  203 non-null    category
12  embark_town           889 non-null    object
13  alive                 891 non-null    object
14  alone                 891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200
survived	0					
pclass	0					
sex	0					

```
age                177
sibsp              0
parch              0
fare               0
embarked           2
class              0
who                0
adult_male         0
deck               688
embark_town         2
alive              0
alone              0
dtype: int64
Training Data Shape: (712, 7)
Test Data Shape: (179, 7)
```

code2:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Load Wine dataset
wine = load_wine()
X, y = wine.data, wine.target

# Scale Features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.25, random_state=101, stratify=y)

# SVM Model with RBF Kernel
svm_wine = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_wine.fit(X_train, y_train)
y_pred_svm = svm_wine.predict(X_test)

# Random Forest Model
rf_wine = RandomForestClassifier(n_estimators=150, max_depth=5, random_state=101)
rf_wine.fit(X_train, y_train)
y_pred_rf = rf_wine.predict(X_test)

print("--- SVM Classification Report ---")
print(classification_report(y_test, y_pred_svm, target_names=wine.target_names))

print("--- Random Forest Classification Report ---")
print(classification_report(y_test, y_pred_rf, target_names=wine.target_names))
```

Output2:

```
... --- SVM Classification Report ---
              precision    recall  f1-score   support

   class_0       1.00      1.00      1.00        15
   class_1       0.94      0.94      0.94        18
   class_2       0.92      0.92      0.92        12

 accuracy              0.96              45
 macro avg           0.95      0.95      0.95              45
 weighted avg        0.96      0.96      0.96              45

--- Random Forest Classification Report ---
              precision    recall  f1-score   support

   class_0       1.00      1.00      1.00        15
   class_1       1.00      0.94      0.97        18
   class_2       0.92      1.00      0.96        12

 accuracy              0.98              45
 macro avg           0.97      0.98      0.98              45
 weighted avg        0.98      0.98      0.98              45
```

RESULT:

The Python environment was successfully set up and the dataset was pre-processed by handling missing values, encoding categorical data, performing feature scaling, and splitting the data into training and testing sets. The dataset is now ready for model training and analysis.